04/04/2026

Phân lớp dữ liệu bệnh tiểu đường

Sử dụng bảng dữ liệu Pima Indians Diabetes

ĐƯỢC THUYẾT TRÌNH BỞI
Văn Nguyễn Đức Thịnh

Thành viên

3124410344 - Văn Nguyễn Đức Thịnh
3124410158 - Lê Đăng Khoa
3124410315 - Phạm Quốc Tân
3124410019 - Trần Gia Bảo
3124410219 - Lê Quốc Nam

Dữ liệu kiểm nghiệm

Phân chia tập dữ liệu (Global Hold-out 7/3)

Tập dữ liệu gốc sau khi trải qua bước tiền xử lý và làm sạch được phân chia thành hai phần độc lập theo tỉ lệ 7/3 (70% cho huấn luyện và 30% cho kiểm thử).

- Tập Train (Huấn luyện): Chứa 482 mẫu. Tập này được sử dụng để thuật toán học các trọng số, đặc trưng sinh lý và tinh chỉnh siêu tham số.

- Tập Test (Kiểm nghiệm): Chứa 238 mẫu. Tập này bị "đóng băng" hoàn toàn trong suốt quá trình xây dựng mô hình và chỉ được sử dụng ở bước đánh giá cuối cùng nhằm đo lường tính hiệu quả thực tế của thuật toán.

Dữ liệu kiểm nghiệm

```
print("\n=== ĐỐI CHIẾU SỐ DÒNG ===")
print(f"Số dòng ban đầu: {rows_before}")
print(f"Số dòng có lỗi (unique rows): {rows_with_error}")
print(f"Số dòng sau khi làm sạch: {rows_after}")
print(f"Số dòng bị loại thực tế: {rows_before - rows_after}")
```

```
=== ĐỐI CHIẾU SỐ DÒNG ===
Số dòng ban đầu: 768
Số dòng có lỗi (unique rows): 48
Số dòng sau khi làm sạch: 720
Số dòng bị loại thực tế: 48
```

Chia dữ liệu thực nghiệm

- Dữ liệu ban đầu: chia 30% dữ liệu dùng để Test, 70% dùng để train
- Dữ liệu train: chia chiến lược K-Fold với k = 5

```
index_all = np.arange(len(df_org))
index_train, index_test = train_test_split(index_all, test_size = 0.33, random_state=seed)
```

```
df_train = df_org.iloc[index_train]
df_test  = df_org.iloc[index_test]
print(f'train: {len(df_train)} / test: {len(df_test)} - Tỷ lệ: {len(df_train) / len(df_org)}, {len(df_test) / len(df_org)}')
```

```
train: 482 / test: 238 - Tỷ lệ: 0.6694444444444444, 0.33055555555555555
```

Dữ liệu kiểm nghiệm

Chiến lược huấn luyện và điều chỉnh tham số nội bộ

Để đánh giá và tinh chỉnh tham số (Hyperparameter tuning) cho các mô hình (như SVM, K-Neighbors, Decision Tree,...) mà không làm rò rỉ thông tin từ tập Test, tập Train (482 mẫu) buộc phải được cắt nhỏ thêm một cấp độ phân rã nữa thông qua các chiến lược lượng giá nội bộ: Hold-out phân cấp và Kiểm chứng chéo K-Fold.

Dữ liệu kiểm nghiệm

Chiến lược Hold-out nội bộ (Train/Valid Split)

Tập Train (482 mẫu) tiếp tục được chia nhỏ theo tỉ lệ 7/3 để tạo ra một tập kiểm định (Validation set) chuyên biệt. Sau quá trình xử lý nhiễu nội bộ, cấu trúc tập dữ liệu thực tế thu được lưu trong file holdout_idx.npz như sau:

· Tập Train nội bộ (Train sub-set): 337 mẫu (dùng để khớp mô hình).

· Tập Validation (Valid set): 145 mẫu (dùng để đánh giá và hiệu chỉnh tham số nhanh chóng).

· Tổng cộng có 482 mẫu (lọc đi 32 hồ sơ).

Dữ liệu kiểm nghiệm

```python
# Tách feature và label

X_train = df_train.iloc[:, :-1]
y_train = df_train.iloc[:, -1]

X_test  = df_test.iloc[:, :-1]
y_test  = df_test.iloc[:, -1]
```

```python
scaler = StandardScaler()

x_train = scaler.fit_transform(X_train)
x_test  = scaler.transform(X_test)
y_train = y_train.values
```

```python
X_train_hold, X_valid_hold, y_train_hold, y_valid_hold = train_test_split(
    x_train,
    y_train,
    test_size=0.3,
    random_state=seed,
    stratify=y_train
)

print("Hold-out split")
print("Train:", X_train_hold.shape)
print("Valid:", X_valid_hold.shape)
```

```
Hold-out split
Train: (337, 8)
Valid: (145, 8)
```

Dữ liệu kiểm nghiệm

Chiến lược K-fold Cross Validation

Mặc dù Hold-out nội bộ cho tốc độ tính toán nhanh, nhưng nó dễ bị ảnh hưởng bởi phương sai dữ liệu nếu cách chia ngẫu nhiên không hội tụ đủ các trường hợp y khoa phức tạp. Để khắc phục, chiến lược K-fold (với k=5 hoặc 10) được áp dụng trên toàn bộ tập Train gốc.

Phương pháp này chia dữ liệu huấn luyện thành k phần (folds) đều nhau. Quá trình mô hình hóa sẽ lặp lại k lần, mỗi lần lấy k-1 phần làm dữ liệu Train và 1 phần còn lại làm Validation. Điểm số đánh giá cuối cùng là trung bình cộng của k lần kiểm định.

Dữ liệu kiểm nghiệm

k-fold (chia thành k phần đều nhau với k-1 phần cho train/1 phần cho valid)

```
kfold = StratifiedKFold(
    n_splits=10,
    shuffle=True,
    random_state=seed
)

print(kfold)
```

```
StratifiedKFold(n_splits=10, random_state=42, shuffle=True)
```

```
for train_idx, valid_idx in kfold.split(x_train, y_train):

    X_train_k = x_train[train_idx]
    y_train_k = y_train[train_idx]

    X_valid_k = x_train[valid_idx]
    y_valid_k = y_train[valid_idx]

    print("Train:", X_train_k.shape, "Valid:", X_valid_k.shape)
    break
```

```
Train: (433, 8) Valid: (49, 8)
```

Dữ liệu kiểm nghiệm

Tổng kết vai trò quy trình kiểm nghiệm

Sự kết hợp giữa chia tách tổng thể (idx.npz) và chia tách nội bộ (holdout_idx.npz, K-fold) tạo thành một chu trình khép kín:

1. Train set (k-1 folds hoặc 70% nội bộ): Môi trường để thuật toán tìm ra quy luật từ đặc trưng sinh lý học (như mức đường huyết, độ dày biểu bì, BMI,...).
2. Valid set (1 fold hoặc 30% nội bộ): Thước đo tạm thời để tìm ra bộ tham số (parameters) mang lại độ chính xác, độ nhạy cao nhất.
3. Test set (238 mẫu độc lập): Giám khảo cuối cùng. Kết quả đánh giá trên tập này là kết quả chính thức được báo cáo trong ứng dụng chẩn đoán y khoa thực tế (Mục 8). Quy trình này tuân thủ tuyệt đối tiêu chuẩn đánh giá thuật toán máy học hiện đại.
4. Tính sẵn sàng: Dữ liệu đã được đóng gói (feat_minmax.npz, feat_standard.npz) và chuẩn bị đầy đủ cho Bước 7 (Lượng giá thuật toán) và Bước 8 (Kiểm nghiệm kết quả) một cách trơn tru.
5. Tính tái lập (Reproducibility): Toàn bộ trạng thái phân chia của thí nghiệm được lưu trữ chuẩn xác thông qua các ma trận Numpy (idx.npz, holdout_idx.npz), cho phép các nhà nghiên cứu khác có thể dễ dàng kiểm chứng chéo lại kết quả của dự án.

Lượng giá thuật toán (Evaluate Algorithms)

Giới thiệu mô hình máy học

Logistic Regression	Mô hình phân loại tuyến tính sử dụng hàm sigmoid để dự đoán xác suất thuộc về từng lớp.
Linear Discriminant Analysis	Thuật toán tìm không gian chiếu tuyến tính giúp tối đa hóa khoảng cách giữa các lớp dữ liệu.
K-Nearest Neighbors	Phân loại mẫu mới dựa trên nhãn của K điểm dữ liệu gần nhất trong tập huấn luyện.
Decision Tree	Xây dựng cây quyết định bằng cách chia dữ liệu theo các điều kiện để dự đoán nhãn lớp.
Gaussian Naive Bayes	Phân loại dựa trên định lý Bayes với giả định các đặc trưng độc lập và phân phối chuẩn.
Support Vector Machine	Tìm siêu phẳng tối ưu để phân tách các lớp với khoảng cách biên lớn nhất.

Lượng giá thuật toán (Evaluate Algorithms)

```
LR
Accuracy: 0.7720663265306122
Std: 0.04726179843101965
----------------
LDA
Accuracy: 0.7721513605442176
Std: 0.041005641774539366
----------------
KNN
Accuracy: 0.7477891156462585
Std: 0.0716473508091368
----------------
CART
Accuracy: 0.718579931972789
Std: 0.09299013713930986
----------------
NB
Accuracy: 0.753656462585034
Std: 0.056858513172203494
----------------
SVM
Accuracy: 0.7740646258503401
Std: 0.045914902867541164
----------------
```

Logistic Regression và Linear Discriminant Analysis: Độ chính xác tương đối cao và ổn định → Phân tách khá tốt trong không gian tuyến tính

Support Vector Machine: Độ ổn định tốt → Tổng quát hóa tốt, ít bị biến động

K-Nearest Neighbors: Độ lệch chuẩn lớn, độ chính xác ổn → Phụ thuộc nhiều vào dữ liệu, có thể bị biến động

Decision Tree: Độ chính xác thấp nhất, độ lệch chuẩn tương đối lớn → Dễ bị overfitting nhưng khả năng giải thích tốt và tốc độ huấn luyện nhanh

Gaussian Naive Bayes: Độ chính xác cao, độ ổn định ở mức chấp nhận được → Bị hạn chế khi các thuộc tính có mối quan hệ với nhau

Lượng giá thuật toán (Evaluate Algorithms)

Kết luận:

Mô hình Logistic Regression, Linear Discriminant Analysis và Support Vector Machine cho kết quả tốt và ổn định hơn so với các thuật toán còn lại. Trong đó, Support Vector Machine đạt độ chính xác cao nhất và độ lệch chuẩn nhỏ, cho thấy khả năng tổng quát hóa tốt nhất trên tập dữ liệu. Hai thuật toán này được lựa chọn để tinh chỉnh tham số

Lượng giá thuật toán (Evaluate Algorithms)

Tinh chỉnh tham số mô hình K-Nearest Neighbors

Sau khi đánh giá mô hình K-Nearest Neighbors với tham số mặc định, bước tiếp theo là tiến hành tinh chỉnh tham số nhằm cải thiện hiệu suất phân loại. Trong thuật toán KNN, tham số quan trọng nhất là số lượng láng giềng gần nhất K. Giá trị của K ảnh hưởng trực tiếp đến khả năng học của mô hình. Nếu K quá nhỏ, mô hình dễ bị ảnh hưởng bởi nhiễu; nếu K quá lớn, mô hình có thể làm mất đi ranh giới phân tách giữa các lớp.

```
Best score: 0.7763605442176871

Best params: {'n_neighbors': 7}
```

Trước khi tinh chỉnh: độ chính xác ≈ 0.748

Sau khi tỉnh chỉnh, độ chính xác ≈ 0.776

Lượng giá thuật toán (Evaluate Algorithms)

Tinh chỉnh tham số mô hình Support Vector Machine

Sau khi đánh giá các mô hình với tham số mặc định, thuật toán Support Vector Machine cũng được tiến hành tinh chỉnh tham số nhằm cải thiện hiệu quả phân loại. Hiệu suất của SVM phụ thuộc nhiều vào các tham số điều chỉnh, đặc biệt là hệ số phạt C, loại kernel và tham số gamma.

Phương pháp Grid Search kết hợp với K-Fold Cross Validation được sử dụng. Nhiều tổ hợp tham số điều chỉnh được thử nghiệm, mô hình được huấn luyện và đánh giá trên từng cấu hình. Tổ hợp tham số có độ chính xác trung bình cao nhất được lựa chọn làm cấu hình tối ưu cho SVM.

```
Best score: 0.7762755102040816

Best params: {'C': 0.1, 'gamma': 'scale', 'kernel': 'linear'}
```

Trước khi tinh chỉnh: độ chính xác ≈ 0.774

Sau khi tỉnh chỉnh, độ chính xác ≈ 0.776

Lượng giá thuật toán (Evaluate Algorithms)

Huấn luyện theo mô hình k - fold

Để thực hiện việc đánh giá và chọn tham số tối ưu đó, ta sử dụng phương pháp K-Fold Cross Validation

K-Fold Cross Validation là phương pháp chia dữ liệu thành k phần bằng nhau, sau đó lặp k lần huấn luyện, mỗi lần sử dụng một phần làm tập kiểm tra và k−1 phần còn lại làm tập huấn luyện, trong đó mỗi phần đều được dùng làm tập kiểm tra đúng một lần, thường chọn k = 5 hoặc 10 để đảm bảo cân bằng giữa độ chính xác và chi phí tính toán.

```
kfold = StratifiedKFold(
    n_splits=10,
    shuffle=True,
    random_state=seed
)

print(kfold)
```

```
results = []
names = []

for name, model in models:

    scores = cross_val_score(
        model,
        x_train,
        y_train,
        cv=kfold,
        scoring="accuracy"
    )


    results.append(scores)
    names.append(name)

    print(name)
    print("Accuracy:", scores.mean())
    print("Std:", scores.std())
    print("----------------")
```

Lượng giá thuật toán (Evaluate Algorithms)

Huấn luyện theo mô hình k - fold

Tóm lại, việc áp dụng K-Fold Cross Validation giúp nâng cao độ tin cậy trong quá trình huấn luyện và đánh giá mô hình, từ đó tạo ra mô hình có khả năng tổng quát hóa tốt và dự đoán chính xác hơn trên dữ liệu thực tế.

Lượng giá thuật toán (Evaluate Algorithms)

Đánh giá tổng quát mô hình mới

Logistic Regression (Hồi quy Logistic)

K-Nearest Neighbors (K láng giềng gần nhất)

Naive Bayes (Bayes ngây thơ)

```
LR _tuned
Accuracy: 0.7782312925170067
Std: 0.038572182172388336
----------------
LDA _tuned
Accuracy: 0.7721513605442176
Std: 0.041005641774539366
----------------
KNN _tuned
Accuracy: 0.7803146258503401
Std: 0.05479452552839471
----------------
CART _tuned
Accuracy: 0.7454931972789116
Std: 0.07216357344153954
----------------
NB _tuned
Accuracy: 0.753656462585034
Std: 0.056858513172203494
----------------
SVM _tuned
Accuracy: 0.7802721088435374
Std: 0.04298122424883498
----------------
```

Linear Discriminant Analysis (Phân tích biệt thức tuyến tính)

Classification And Regression Tree (Cây quyết định phân loại và hồi quy)

Support Vector Machine (Máy vector hỗ trợ)

Lượng giá thuật toán (Evaluate Algorithms)

Đánh giá tổng quát mô hình mới

- SVM: Accuracy cao nhất (~0.78), độ ổn định tốt → mô hình tốt nhất-
- Logistic Regression (LR): Accuracy gần SVM, ổn định nhất
- LDA: Hiệu suất khá, ổn định trung bình
- KNN: Accuracy cao nhưng dao động lớn → kém ổn định
- Naive Bayes (NB): Hiệu suất trung bình, chưa ổn định
- CART: Accuracy thấp nhất, dễ overfitting

Mô hình SVM là lựa chọn tối ưu nhất do đạt được sự cân bằng tốt giữa độ chính xác và độ ổn định, cho thấy khả năng tổng quát hóa hiệu quả trên dữ liệu thực tế.
Bên cạnh đó, Logistic Regression (LR) cũng là một phương án thay thế đáng tin cậy nhờ có độ ổn định cao và hiệu suất gần tương đương, phù hợp trong trường hợp ưu tiên tính nhất quán của mô hình.

CHIẾN LƯỢC & QUY TRÌNH KIỂM NGHIỆM

Chiến lược kiểm nghiệm trên Tập Test Nội dung

Cách ly dữ liệu tuyệt đối: Khóa két 30% dữ liệu (216 bản ghi) ngay từ đầu để làm tập kiểm thử thực tế, đảm bảo mô hình không bị "học vẹt".
Triệt tiêu Rò rỉ dữ liệu (Data Leakage): Bộ chuẩn hóa (Scaler) thiết lập nguyên tắc chỉ "khớp" trên tập Train và "biến đổi" trên tập Test.
Danh sách tham chiến: 6 mô hình cơ sở (Baselines) và 2 mô hình tinh chỉnh chuyên sâu (Tuned SVM, Tuned KNN).
Hệ thống độ đo: Tập trung vào Accuracy, Precision, F1-Score và đặc biệt ưu tiên Độ nhạy (Recall).

KẾT QUẢ ĐÁNH GIÁ CHUNG CUỘC

Kết quả Kiểm nghiệm Thực tế

```
============================================================
KẾT QUẢ ĐÁNH GIÁ TRÊN TẬP TEST (MỤC 8)
============================================================
 > Logistic Regression (Base)  : Accuracy = 0.7685 (76.85%)
 > LDA (Base)                  : Accuracy = 0.7639 (76.39%)
 > Decision Tree (Base)        : Accuracy = 0.6852 (68.52%)
 > Naive Bayes (Base)          : Accuracy = 0.7546 (75.46%)
 > KNN (Base)                  : Accuracy = 0.7222 (72.22%)
 > SVM (Base)                  : Accuracy = 0.7778 (77.78%)
 > KNN (Tuned)                 : Accuracy = 0.7500 (75.00%)
 > SVM (Tuned)                 : Accuracy = 0.7778 (77.78%)
------------------------------------------------------------
MÔ HÌNH XUẤT SẮC NHẤT: SVM (Base) với Accuracy = 77.78%

[BÁO CÁO CHI TIẾT MÔ HÌNH TỐT NHẤT]
              precision    recall  f1-score   support

Khỏe mạnh (0)       0.79      0.89      0.84       142
 Mắc bệnh (1)       0.73      0.55      0.63        74

     accuracy                           0.78       216
    macro avg       0.76      0.72      0.74       216
 weighted avg       0.77      0.78      0.77       216
```

Mô hình xuất sắc nhất: Support Vector Machine (SVM) vươn lên dẫn đầu với độ chính xác tổng thể nhảy vọt lên mức 77.78%.

Cột mốc học thuật: Chính thức xô đổ ngưỡng lịch sử 76% của thuật toán mạng nơ-ron thích nghi ADAP (Smith và cộng sự, 1988).

Sự hội tụ tham số: Cả phiên bản SVM (Base) và SVM (Tuned) đều đạt chung mức 77.78%, minh chứng cho sức mạnh vượt trội của không gian nhân đa chiều (kernel) khi xử lý dữ liệu y sinh của người Pima.

GÓC NHÌN Y TẾ & HƯỚNG CẢI TIẾN

Ý nghĩa Lâm sàng & Bài toán "Âm tính giả"

Cạm bẫy của Accuracy: Dù xô đổ kỷ lục lịch sử, nhưng khi nhìn vào ma trận nhầm lẫn, Độ nhạy (Recall) của nhóm Mắc bệnh chỉ đạt mức 55%.

Góc nhìn khốc liệt: Hệ thống được điểm cao chủ yếu do nhận diện quá tốt nhóm người khỏe mạnh (Recall 89%), nhưng thực tế vẫn đang bỏ lọt tới 45% bệnh nhân mang bệnh.

Định hướng bắt buộc: Trong y tế dự phòng tầm soát đầu tuyến, không thể chỉ chạy theo Độ chính xác tổng thể. Cần chủ động hạ ngưỡng chẩn đoán hoặc phạt trọng số để ép thuật toán ưu tiên tối đa hóa độ nhạy.

SẴN SÀNG CHO MÔI TRƯỜNG TRIỂN KHAI

Đóng gói hệ thống: Trích xuất thành công lõi thuật toán tối ưu (SVM) và bộ lọc chuẩn hóa dữ liệu thành các khối độc lập sẵn sàng chuyển giao.

Hỗ trợ quyết định lâm sàng: Sẵn sàng tích hợp trực tiếp vào hệ thống phần mềm Quản lý Hồ sơ Bệnh án Điện tử (EHR) tại các cơ sở y tế.

Tự động hóa tầm soát: Hoạt động như một trợ lý y khoa ngầm, tự động tính toán xác suất bệnh lý và phát cờ cảnh báo (flagging) ngay khi nhân viên y tế nhập sinh hiệu của bệnh nhân.